Independent University, Bangladesh

CSE313 Compiler Construction · Practice Set C · Mid + Quiz · Time: 1h30m

New questions. Q1 is NOT ambiguous (like Aut-23). Q2 is NOT LL(1). Q3 is a reason question (like Summer-23).

Question paper

Mid Question (Total marks: 20)

1. Prove whether the following grammar is Ambiguous or NOT. If the grammar is ambiguous, convert it to unambiguous form. **[7]**

```
E → E $ T | T
T → F % T | F
F → F ! | ( E ) | n
```

2. Write the First() and Follow() of the following Grammar. Draw LL(1) Parsing Table and parse the string `a a b` showing all steps. If parsing is not possible, point out the reasons. **[8]**

```
S → A a B | b C
A → a A | ε
B → b | ε
C → c C | d
```

3. A student removed left recursion from an ambiguous grammar and claimed the new grammar is unambiguous. Is the claim correct? Justify with an example. **[5]**

Quiz Question (Total marks: 10)

1. Convert the following Non-Deterministic Grammar (NDG) to Deterministic Grammar (DG): **[5]**

```
X → p q r X | p q r Y | p q | p s
Y → y
```

2. Convert the following Left Recursive Grammar (LRG) to Right Recursive Grammar (RRG): **[5]**

```
P → P + Q | P - Q | Q
Q → Q * id | id
```

Answers (exam layout, with comments)

Answer 1 [7]

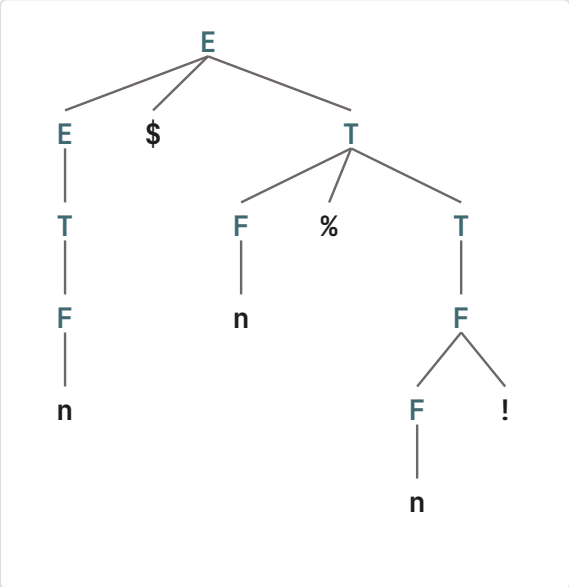
`WRITE THIS`

Check every rule for the shape X → X op X:

Rule	Recursion
E → E $ T	left only
T → F % T	right only
F → F !	left only (postfix)

Each operator lives on one level and recurses on one side, so precedence and associativity are fixed. Example `n $ n % n !`: only one rule fits at each step.

LMD: E ⇒ E $ T ⇒ T $ T ⇒ F $ T ⇒ n $ T ⇒ n $ F % T ⇒ n $ n % T ⇒ n $ n % F ⇒ n $ n % F ! ⇒ n $ n % n !



Every string has exactly one leftmost derivation parse tree ∴ the grammar is **NOT ambiguous**. No conversion is needed.

Operator	Precedence	Associativity
$	1 (lowest)	left (E → E $ T)
%	2	right (T → F % T)
!	3 (highest)	postfix, left (F → F !)

Checked by computer: all 207 strings up to length 7 have exactly one LMD.

> **Lose marks by:** calling it ambiguous because it is left recursive (left recursion is not ambiguity).

Answer 2 [8]

WRITE THIS

Numbered rules: 1. S → A a B 2. S → b C 3. A → a A 4. A → ε 5. B → b 6. B → ε 7. C → c C 8. C → d

Step 1 · FIRST & FOLLOW

Non-terminal	FIRST	FOLLOW
S	{ b, a }	{ $ }
A	{ a, ε }	{ a }
B	{ b, ε }	{ $ }
C	{ c, d }	{ $ }

Step 2 · LL(1) parsing table

	a	b	c	d	$
S	S→AaB	S→bC			
A	A→aA A→ε				
B		B→b			B→ε
C			C→cC	C→d	

M[A, a] has A→aA and A→ε. A cell has more than one rule ∴ the grammar is **NOT LL(1)**. The parser cannot choose with one look-ahead, so parsing is not possible.

Why: this is a FIRST/FOLLOW conflict. A → a A puts a rule under FIRST = a. A → ε puts a rule under FOLLOW(A) = { a } (in S → A a B, a comes after A). Same cell, two rules. With LA = a the parser cannot know whether this a belongs to A or is the a after A. ∴ the string `a a b` cannot be parsed by LL(1).

▼ How the FIRST / FOLLOW were found (rough work, not needed on paper)

- `FIRST(S): S → A a B gives { a } (A can be ε → look at the next symbol). S → b C gives { b }. Total { b, a }.`
- `FIRST(A): A → a A gives { a }. A → ε gives ε. Total { a, ε }.`
- `FIRST(B): B → b gives { b }. B → ε gives ε. Total { b, ε }.`
- `FIRST(C): C → c C gives { c }. C → d gives { d }. Total { c, d }.`
- `FOLLOW(S): start symbol → add $. Total { $ }.`
- `FOLLOW(A): in S → A a B: after A is a B → add { a }. Total { a }.`
- `FOLLOW(B): in S → A a B: nothing after B → add FOLLOW(S). Total { $ }.`
- `FOLLOW(C): in S → b C: nothing after C → add FOLLOW(S). Total { $ }.`

> **Lose marks by:** filling the table and parsing anyway; the mark is for pointing at the cell with two rules and naming the conflict.

Answer 3 [5]

WRITE THIS

No, the claim is wrong. Take `E → E + E | id`. It is ambiguous: `id + id + id` has two LMDs. Remove left recursion (α = + E, β = id):

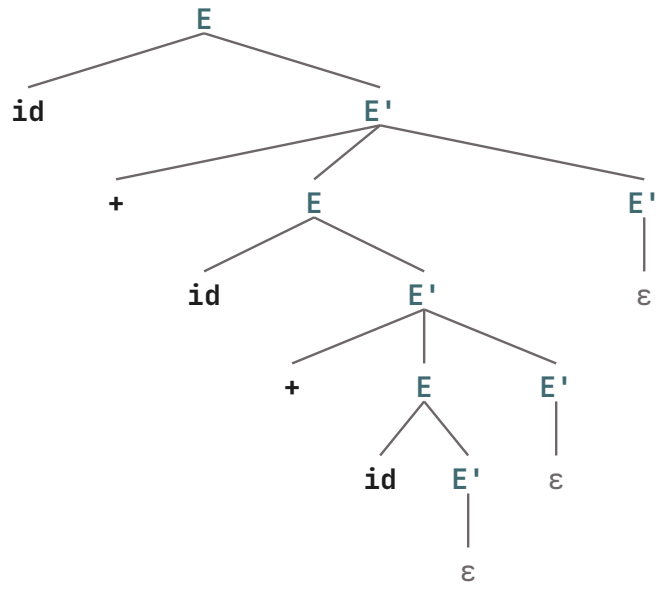
```
E → id E'
E' → + E E' | ε
```

The same string still has two leftmost derivations in the new grammar:

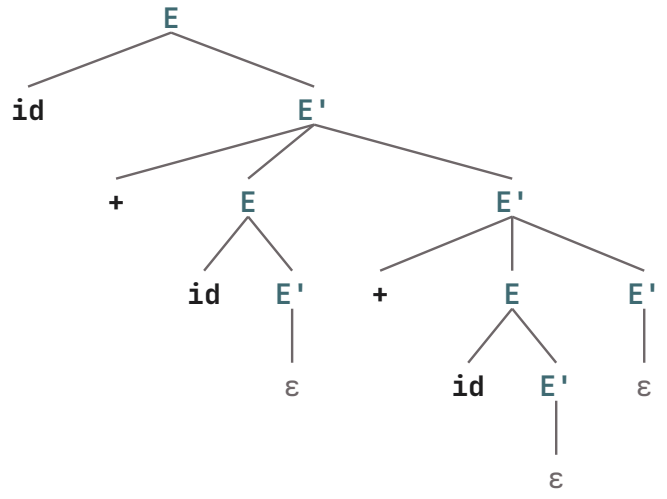
LMD 1: E ⇒ id E' ⇒ id + E E' ⇒ id + id E' E' ⇒ id + id + E E' E' ⇒ id + id + id E' E' E' ⇒ id + id + id E' E' ⇒ id + id + id E' ⇒ id + id + id

LMD 2: E ⇒ id E' ⇒ id + E E' ⇒ id + id E' E' ⇒ id + id E' ⇒ id + id + E E' ⇒ id + id + id E' E' ⇒ id + id + id E' ⇒ id + id + id

Tree 1



Tree 2



Reason: left recursion removal keeps the language and only changes the direction of recursion. It does not add precedence or associativity. Ambiguity comes from the X → X op X shape; here E' → + E E' still lets the second + attach in two places. ∴ To remove ambiguity, rebuild the grammar with precedence levels.

> **Lose marks by:** a claim with no derivations; showing derivations only for the old grammar.

Quiz Answer 1 [5]

`WRITE THIS`

Round 1: in X, `p q r X`, `p q r Y`, `p q`, `p s` all start with `p`. Take it out once:

```
X → p X'
X' → q r X | q r Y | q | s
```

Round 2: in X', `q r X`, `q r Y`, `q` all start with `q`. Take it out once:

```
X' → q X'' | s
X'' → r X | r Y | ε
```

Round 3: in X'', `r X`, `r Y` all start with `r`. Take it out once:

```
X'' → r X''' | ε
X''' → X | Y
```

Final DG:

```
X → p X'
X' → q X'' | s
X'' → r X''' | ε
X''' → X | Y
Y → y
```

> **Lose marks by:** this needs three rounds; check every new rule again.

Quiz Answer 2 [5]

`WRITE THIS`

P → P + Q | P - Q | Q: α = `+ Q` , `- Q`, β = `Q`.

```
P → Q P'
P' → + Q P' | - Q P' | ε
```

Q → Q * id | id: α = `* id`, β = `id`.

```
Q → id Q'
Q' → * id Q' | ε
```

Final RRG:

```
P → Q P'
P' → + Q P' | - Q P' | ε
Q → id Q'
Q' → * id Q' | ε
```

Rule: A → Aα | β ⇒ A → βA', A' → αA' | ε. "β goes first, α goes to the new guy, new guy ends with ε."